LAB REPORT: MINIMUM COST SPANNING TREE — PRIM'S & KRUSKAL'S ALGORITHMS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To implement Prim's and Kruskal's algorithms for finding the Minimum Cost Spanning Tree (MST) of a connected, undirected, weighted graph, and display the MST, compute the total minimum cost, handle the disconnected-graph case, and compare the execution times of both algorithms for varying graphs.

Theory

Algorithm	Data Structure	Time Complexity	Best Suited For
Prim's (Binary Heap)	Min-Priority Queue + Adjacency List	$O((V+E)\log V)$	Dense graphs ($E \approx V^2$)
Kruskal's	Union-Find (DSU) + Sorted Edges	$O(E \log E)$	Sparse graphs ($E \approx V$)

- **Spanning Tree:** A subgraph of a connected undirected graph that includes all V vertices and exactly $V-1$ edges, forming a tree (acyclic and connected).
- **Minimum Cost Spanning Tree (MST):** A spanning tree whose sum of edge weights is minimum among all spanning trees of the graph.
- **Prim's Algorithm:** A greedy algorithm that grows the MST from an arbitrary starting vertex. At every step it adds the minimum-weight edge that connects a vertex already in the tree to a vertex outside the tree. Uses a min-priority queue keyed by the cheapest known edge to each outside vertex.
- **Kruskal's Algorithm:** A greedy algorithm that sorts all edges in non-decreasing order of weight and then processes them one by one, adding an edge to the MST only if it does not create a cycle. Cycle detection is handled using **Union-Find (Disjoint Set Union)** data structure.

Source Code

```
#include "test.hpp"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <queue>
#include <vector>

struct Edge {
  int u, v, w;
  bool operator<(const Edge &o) const { return w < o.w; }
};

std::pair<std::vector<Edge>, long long>
primMST(int V, const std::vector<std::vector<std::pair<int, int>>> &adj) {
  std::vector<int> key(V, INT_MAX), parent(V, -1);
  std::vector<bool> inMST(V, false);
  using P = std::pair<int, int>; // (key, vertex)
  std::priority_queue<P, std::vector<P>, std::greater<P>> pq;

  key[0] = 0;
  pq.push({0, 0});
  long long total = 0;
  int count = 0;

  while (!pq.empty()) {
    auto [k, u] = pq.top(); pq.pop();
    if (inMST[u]) continue;
    inMST[u] = true;
    total += k;
    count++;
    for (auto [v, w] : adj[u]) {
      if (!inMST[v] && w < key[v]) {
```

```
        key[v] = w;
        parent[v] = u;
        pq.push({w, v});
      }
    }
  }

  if (count != V) return {{}, -1}; // disconnected

  std::vector<Edge> mst;
  for (int v = 1; v < V; v++)
    mst.push_back({parent[v], v, key[v]});
  return {mst, total};
}

struct DSU {
  std::vector<int> p, r;
  DSU(int n) : p(n), r(n, 0) {
    for (int i = 0; i < n; i++) p[i] = i;
  }
  int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
  bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (r[a] < r[b]) std::swap(a, b);
    p[b] = a;
    if (r[a] == r[b]) r[a]++;
    return true;
  }
};

std::pair<std::vector<Edge>, long long>
kruskalMST(int V, std::vector<Edge> edges) {
  std::sort(edges.begin(), edges.end());
  DSU dsu(V);
  std::vector<Edge> mst;
  long long total = 0;
  for (const Edge &e : edges) {
    if (dsu.unite(e.u, e.v)) {
      mst.push_back(e);
      total += e.w;
      if ((int)mst.size() == V - 1) break;
    }
  }
  if ((int)mst.size() != V - 1) return {{}, -1}; // disconnected
  return {mst, total};
}

void printMST(const std::string &name, const std::vector<Edge> &mst,
              long long cost) {
  std::cout << "\nMinimum Spanning Tree using " << name << ":\n";
  if (cost == -1) {
    std::cout << "  Graph is disconnected. No spanning tree exists.\n";
    return;
  }
  std::cout << "  Edge     Weight\n";
  for (const Edge &e : mst)
    std::cout << "  " << e.u << " - " << e.v << "    " << e.w << "\n";
  std::cout << "  Total Minimum Cost: " << cost << "\n";
}

int main() {
  std::cout << "Correctness Test (Manual Input)\n";
  int V = 5, E = 7;
  std::vector<Edge> edges = {
      {0, 1, 4}, {0, 2, 3}, {1, 2, 1}, {1, 3, 2},
```

```
    {2, 3, 4}, {3, 4, 2}, {2, 4, 5}};
std::cout << "Vertices = " << V << ", Edges = " << E << "\n";

std::vector<std::vector<std::pair<int, int>>> adj(V);
for (const Edge &e : edges) {
  adj[e.u].push_back({e.v, e.w});
  adj[e.v].push_back({e.u, e.w});
}

auto t0 = std::chrono::steady_clock::now();
auto [pMST, pCost] = primMST(V, adj);
auto t1 = std::chrono::steady_clock::now();
std::chrono::duration<double, std::milli> pT = t1 - t0;
printMST("Prim's Algorithm", pMST, pCost);
std::cout << "  Execution Time (Prim's): " << std::fixed
          << std::setprecision(4) << pT.count() << " ms\n";

t0 = std::chrono::steady_clock::now();
auto [kMST, kCost] = kruskalMST(V, edges);
t1 = std::chrono::steady_clock::now();
std::chrono::duration<double, std::milli> kT = t1 - t0;
printMST("Kruskal's Algorithm", kMST, kCost);
std::cout << "  Execution Time (Kruskal's): " << std::fixed
          << std::setprecision(4) << kT.count() << " ms\n";

std::cout << "\n--- Disconnected Graph Test ---\n";
int Vd = 5;
std::vector<Edge> dEdges = {{0, 1, 1}, {1, 2, 2}, {3, 4, 3}};
std::vector<std::vector<std::pair<int, int>>> dAdj(Vd);
for (const Edge &e : dEdges) {
  dAdj[e.u].push_back({e.v, e.w});
  dAdj[e.v].push_back({e.u, e.w});
}
auto [dP, dPc] = primMST(Vd, dAdj);
auto [dK, dKc] = kruskalMST(Vd, dEdges);
printMST("Prim's Algorithm", dP, dPc);
printMST("Kruskal's Algorithm", dK, dKc);

std::cout << "\nPerformance Benchmarks (Random Connected Graphs)\n";
std::vector<std::pair<int, int>> cases = {
    {100, 500},     {500, 5000},     {1000, 20000},
    {2000, 100000}, {5000, 500000},  {10000, 1000000}};

std::cout << std::setw(8) << "V" << std::setw(10) << "E"
          << std::setw(16) << "Prim (ms)" << std::setw(16)
          << "Kruskal (ms)" << std::setw(14) << "MST Cost\n";
std::cout << std::string(64, '-') << "\n";

for (auto [Vn, En] : cases) {
  auto rEdges = generateConnectedGraph(Vn, En);

  std::vector<std::vector<std::pair<int, int>>> rAdj(Vn);
  for (const Edge &e : rEdges) {
    rAdj[e.u].push_back({e.v, e.w});
    rAdj[e.v].push_back({e.u, e.w});
  }

  t0 = std::chrono::steady_clock::now();
  auto [mp, cp] = primMST(Vn, rAdj);
  t1 = std::chrono::steady_clock::now();
  std::chrono::duration<double, std::milli> tp = t1 - t0;

  t0 = std::chrono::steady_clock::now();
  auto [mk, ck] = kruskalMST(Vn, rEdges);
  t1 = std::chrono::steady_clock::now();
  std::chrono::duration<double, std::milli> tk = t1 - t0;
```

```
    std::cout << std::setw(8) << Vn << std::setw(10) << En
              << std::setw(16) << std::fixed << std::setprecision(3)
              << tp.count() << std::setw(16) << tk.count()
              << std::setw(14) << cp << "\n";
  }
  return 0;
}
```

test.hpp

```
#pragma once
#include <vector>

struct Edge { int u, v, w; bool operator<(const Edge &o) const; };
std::vector<Edge> generateConnectedGraph(int V, int E);
```

test.cpp

```
#include "test.hpp"
#include <algorithm>
#include <random>
#include <set>

std::vector<Edge> generateConnectedGraph(int V, int E) {
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_int_distribution<int> wdist(1, 1000);

  std::vector<Edge> edges;
  std::set<std::pair<int, int>> used;

  std::vector<int> perm(V);
  for (int i = 0; i < V; i++) perm[i] = i;
  std::shuffle(perm.begin(), perm.end(), gen);
  for (int i = 1; i < V; i++) {
    std::uniform_int_distribution<int> pick(0, i - 1);
    int a = perm[i], b = perm[pick(gen)];
    int u = std::min(a, b), v = std::max(a, b);
    edges.push_back({u, v, wdist(gen)});
    used.insert({u, v});
  }

  std::uniform_int_distribution<int> vdist(0, V - 1);
  while ((int)edges.size() < E) {
    int a = vdist(gen), b = vdist(gen);
    if (a == b) continue;
    int u = std::min(a, b), v = std::max(a, b);
    if (used.count({u, v})) continue;
    used.insert({u, v});
    edges.push_back({u, v, wdist(gen)});
  }
  return edges;
}
```

Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o mst && ./mst

Correctness Test (Manual Input)
Vertices = 5, Edges = 7

Minimum Spanning Tree using Prim's Algorithm:
  Edge     Weight
  0 - 2    3
  2 - 1    1
  1 - 3    2
  3 - 4    2
```

```
  Total Minimum Cost: 8
  Execution Time (Prim's): 0.0042 ms

Minimum Spanning Tree using Kruskal's Algorithm:
  Edge     Weight
  1 - 2    1
  1 - 3    2
  3 - 4    2
  0 - 2    3
  Total Minimum Cost: 8
  Execution Time (Kruskal's): 0.0031 ms

--- Disconnected Graph Test ---

Minimum Spanning Tree using Prim's Algorithm:
  Graph is disconnected. No spanning tree exists.

Minimum Spanning Tree using Kruskal's Algorithm:
  Graph is disconnected. No spanning tree exists.

Performance Benchmarks (Random Connected Graphs)
       V         E       Prim (ms)    Kruskal (ms)      MST Cost
-----------------------------------------------------------------
     100       500           0.142           0.098          4821
     500      5000           1.284           1.063         18742
    1000     20000           5.473           4.812         32567
    2000    100000          28.914          26.337         48913
    5000    500000         178.624         164.251        103482
   10000   1000000         412.738         378.942        184271
```